MATHÉMATIQUES ET INFORMATIQUE
Sciences
Université Paris Cité

2026

CAHIER DE RECETTE

L2F2

DAVID JANISZEK

UNIVERSITE PARIS CITE

MATHÉMATIQUES ET INFORMATIQUE
Sciences
Université Paris Cité

Cahier de recette
L2F2

Les informations d'identification du projet

Référence du document :	CDR-L2F2
Version du document :	1.0
Date du document :	18/04/26
Auteur(s) :	KANGNI Dina ZOUAD Mohammed

Les éléments de vérification du document

Validé par :	Gaël MAHE
Validé le :	18/04/2026
Soumis le :	18/4/26
Type de diffusion :	Document électronique (.odt)
Confidentialité :	Réservé aux étudiants UFR Maths-Info de l'université Paris Descartes

Les éléments d'authentification

Maître d'ouvrage :	Gaël MAHE
Date / Signature :	

Chef de projet :	Abdallah BENALI
Date / Signature :	

Mots clés : *quantification, déquantification, arbre, cahier de recette, Julia, série temporelle, élagage*

Sommaire

1. **Introduction** 4
   - **1.1. Objectifs et méthodes** 4
   - **1.2. Documents de référence** 4
2. **Guide de lecture** 5
   - **2.1. Maitrise d'œuvre** 5
   - **2.2. Maîtrise d'ouvrage** 5
3. **Concepts de base** 6
4. **Description de la fourniture** 6
5. **Moyen d'essai et outils** 7
6. **Conformité aux spécifications générales** 7
7. **Conformité aux spécifications fonctionnelles** 7
   - **7.1. Pour chaque scénario** 8
     - **7.1.1. Identification** 8
     - **7.1.2. Description** 8
     - **7.1.3. Contraintes** 8
     - **7.1.4. Critères de validation** 8
     - **7.1.5. Dépendances** 9
8. **Conformité aux spécifications d'interfaces** 10
   - **8.1. Objectifs des tests** 11
   - **8.2. Tests à réaliser** 11
   - **8.3. Critères d'acceptation** 12
9. **Conformité de la documentation** 12
10. **Critère de validation finale** 12
11. **Glossaire** 13
12. **Références** 14
13. **Index** 14

1. Introduction

Ce document est un cahier de recette réalisé pour la conception du projet L2F qui sera développé par la deuxième équipe.

Cette équipe est composée de ACHAK Salim, BENALI Abdallah, KANGNI Dina et ZOUAD Mohammed.

KANGNI Dina et ZOUAD Mohammed l'ont entièrement rédigé sous la direction de Gaël MAHE qui en assure la relecture finale et l'approuve.

1.1. Objectifs et méthodes

Ce document décrit les tests que nous allons effectuer dans le cadre du développement de notre application de déquantification.
Ces tests nous permettent de :

- vérifier que l'application fonctionne correctement, c'est-à-dire qu'elle ne s'interrompt pas au cours de son utilisation,
- vérifier que l'algorithme est bien en train de reconstruire les données en se basant sur les fichiers définis dans le cahier des charges (jeu de données sous-quantifiée xQ et histogramme P),
- vérifier que l'algorithme réduit correctement le nombre de solutions.

L'objectif final de ces tests est de nous assurer que l'application respecte toutes les contraintes et tous les besoins définis par la maitrise d'ouvrage dans le cahier des charges.
Les tests sont organisés à l'aide du module *Test* de Julia. Chaque module dispose de son propre fichier de test regroupés dans un fichier *runtests.jl* qui permet de tous les exécuter et de vérifier leur fonctionnement au sein de l'ecosysteme.

1.2. Documents de référence

Afin de rédiger ce cahier de recette, nous nous sommes appuyés de plusieurs documents :
Le cahier des charges L2F2, qui définit les besoins et les contraintes de l'application et la documentation officielle du langage de programmation **Julia**.

2. Guide de lecture

Dans cette section, nous allons détailler la manière dont les différents acteurs participant à ce projet peuvent utiliser ce document.

2.1. Maitrise d'œuvre

La maitrise d'œuvre est assurée par la deuxième équipe du projet L2F. Elle est composée de quatre membres : KANGNI Dina, BENALI Abdallah, ACHAK Salim et ZOUAD Mohammed.

Elle utilise ce document pour :

- comprendre les tests à réaliser,
- vérifier le bon fonctionnement de l'application,
- détecter et corriger les anomalies,
- valider la conformité technique avant la livraison.

Ce document lui sert donc de guide pour exécuter les tests et garantir la qualité de l'application.

2.2. Maîtrise d'ouvrage

La maitrise d'ouvrage est composée de l'encadrant du projet, Gaël MAHE, ainsi que du responsable de l'UE projet, David JANISZEK.

Elle utilise ce document pour :

- vérifier que l'application répond aux exigences du cahier des charges ;
- valider les résultats obtenus,
- s'assurer que les objectifs scientifiques sont atteints,
- valider la réception finale du projet.

Ce document lui permet ainsi d'évaluer si le projet est conforme aux attentes.

3. Concepts de base

L'objectif de ce projet est de tester un algorithme de déquantification.

La quantification consiste à transformer des données continues en données discrètes, généralement dans un souci d'optimisation de l'espace de stockage.
Le projet que nous allons réaliser se concentre plutôt sur la sous-quantification à 1 bit.
La sous-quantification intervient sur des données qui ont déjà été quantifiées une fois c'est-à-dire qu'on les discrétise une deuxième fois.
Cette opération consiste à supprimer le bit de poids faible divisant, par deux l'ensemble des valeurs possibles.

La sous-quantification (et de manière plus large la quantification) est irréversible : il n'est pas possible de retrouver précisément les données initiales.
L'objectif de notre projet est donc d'implémenter un algorithme qui tenterait de reconstituer un jeu de données après l'opération : c'est ce qu'on appelle la déquantification.

L'idée générale de l'algorithme que nous allons implémenter est d'exploiter l'histogramme P des couples de valeurs successives d'une série temporelle x sous-quantifiée. Cet histogramme nous servira à construire un arbre binaire représentant toutes les séries possibles avant la quantification. On élaguera progressivement l'arbre au fur et à mesure de l'exploitation de l'histogramme, de sorte qu'à la fin l'arbre soit compatible avec lui.

4. Description de la fourniture

L'application livrée comprend :

- un programme écrit en Julia, intégrant un algorithme de déquantification,
- une interface graphique permettant la visualisation de la construction et de l'élagage de l'arbre,
- une suite de tests exécutables via le fichier *runtests.jl et,*
- une documentation utilisateur comprenant, un fichier décrivant la structure du projet et un README.
- Des scripts de lancement et d'installation compatibles avec Windows et Linux.

5. Moyen d’essai et outils

Les tests sont réalisés sur tout ordinateur disposant d’au minimum 4 Go de RAM, quel que soit le système d’exploitation et de la version 1.10.11 de *Julia* présente dans les variables d’environnement.
Une série de jeux de données au format binaire de différentes tailles (c’est-à-dire ayant 10, 100, 200, 500, 999, 9999, 99994, 940, 9980 et 998822 valeurs) ainsi qu’un histogramme composent nos données de tests.
Etant tous codés sur 16 bits, la maitrise d’œuvre sera tenue de développer l’application en tenant compte de cette contrainte.

Ce large éventail de jeu de données fourni permet d’une part, de vérifier visuellement que le programme respecte le cahier des charges (en utilisant les séries plus petites) et par conséquent d’utiliser les plus grandes données de test afin de valider le passage à l'échelle.

6. Conformité aux spécifications générales

Au total, le programme est composé de quatre scripts répartis par système d’exploitation.

En effet, l’utilisateur doit d’abord installer les dépendances via un script d’installation (install.bat pour les utilisateurs de Windows et install.sh pour les utilisateurs de Linux).

Il pourra ensuite lancer l’application via run.bat (compatible avec Windows) ou run.sh (compatible avec Linux).

Ces différentes opérations doivent se lancer sans plantage et doivent mener à l’ouverture et l’exploitation de l’application.

Elle doit pouvoir bien naviguer avec les explorateurs de fichiers relatifs aux systèmes d’exploitation c’est-à-dire charger les fichiers ou dossiers sélectionnés par l’utilisateur.

7. Conformité aux spécifications fonctionnelles

Nous nous assurons que le logiciel répond aux besoins définis dans le cahier des charges. Notre application doit donc charger les données, construire l’arbre des solutions, élaguer les branches incompatibles et reconstruire le signal original.
Ces tests vérifient également la cohérence des résultats produits par l’algorithme.

7.1. Pour chaque scénario

7.1.1. Identification

Chaque scénario associe un identifiant unique numéroté de 1 à 2.

7.1.2. Description

Chaque scénario décrit un test fonctionnel. Il présente l'objectif du test ainsi que les fichiers utilisés en entrée et les fichiers attendus en sortie. On précisera les critères de validation ainsi que les contraintes.

7.1.3. Contraintes

Pour chaque scénario sera précisé les conditions d'exécution des codes tests. On y précisera le format des données et la configuration de notre application.

7.1.4. Critères de validation

Pour chaque scénario sera précisé les critères qui nous confirmerons que les tests ont réussi.

Présentons maintenant les scénarios de test.

Id	Programme testé	Entrée	Contraintes	Sortie attendue	Critères de validation
1	**Programme auxiliaire (*Préparation des données)***	Une série temporelle x .	Fichier binaire encodé sur 16 bits. Version 1.10.11 de Julia installée.	Sa série sous-quantifiée à 1 bit xQ ainsi que son histogramme P.	P et xQ cohérents avec x. Le fichier xQ sauvegardé dans un fichier binaire encodé sur 16 bits et P sauvegardé dans un fichier binaire sous forme de dictionnaire.
2	**Programme principal (Construction et élagage de l'arbre)**	La série sous-quantifiée à 1 bit xQ ainsi que son histogramme P.	P et xQ cohérents avec x. Le fichier xQ sauvegardé dans un fichier binaire encodé sur 16 bits et P sauvegardé dans un fichier binaire sous forme de dictionnaire. Programme auxiliaire exécuté au préalable. Version 1.10.11 de Julia installée.	Tous les x trouvés parmi les branches restantes de l'arbre après son élagage, sauvegardés temporairement dans un dossier *temp/*. L'utilisateur peut sélectionner le nombre de fichiers qu'il souhaite télécharger parmi les solutions produites.	Toutes les séries trouvées sont sauvegardées dans des fichiers binaires encodés sur 16 bits. La série originale x doit figurer parmi toutes les séries candidates trouvées. Si l'utilisateur décide de sélectionner tous les fichiers produits, une archive ZIP est générée.

7.1.5. Dépendances

Les dépendances du projet sont gérées via le fichier *Project.toml* présent à la racine du projet. L'application utilise les bibliothèques suivantes : *GLMakie* pour l'interface graphique et ZipFile pour la génération d'archives ZIP.

L'utilisateur exécute le script *bash* s'il dispose de Linux/Mac ou le script *bat* s'il dispose de Windows.
Ces scripts installent les dépendances via le fichier de configuration et lancent l'application.

Enfin, l'utilisateur doit au préalable avoir installé *Julia* (version 1.10.11), pour pouvoir exécuter l'application.

Le fichier de configuration se présente donc comme suit :

```
n name = "L2F2_Dequantification_App"

uuid = "4dcdeae6-8727-46cb-b2e5-634683e5604d"

version = "1.0.0"

authors = ["KANGNI Dina", "BENALI Abdallah", "ACHAK Salim", "ZOUAD
Mohammed"]


[deps]

GLMakie = "e9467ef8-e4e7-5192-8a1a-b1aee30e663a"

GraphMakie = "1ecd5474-83a3-4783-bb4f-06765db800d2"

Graphs = "86223c79-3864-5bf0-83f7-82e725a168b6"

NativeFileDialog = "e1fe445b-aa65-4df4-81c1-2041507f0fd4"

ZipFile = "a5390f91-8eb1-5f08-bee0-b1d1ffed6cea"


[compat]

GLMakie = "0.10"

GraphMakie = "0.5.10"

Graphs = "1.13.1"

NativeFileDialog = "0.2.1"

ZipFile = "0.10"

julia = "1.10"
```

8. Conformité aux spécifications d’interfaces

L’interface graphique doit permettre à l’utilisateur de charger des données, visualiser l’arbre des solutions et consulter les résultats de reconstruction.

8.1. Objectifs des tests

Les tests d'interface visent à vérifier que l'affichage de l'arbre est lisible et suffisamment espacé, même lorsqu'il contient un grand nombre de branches. Ils permettent également de s'assurer que l'animation est assez lente pour que l'utilisateur puisse suivre l'évolution de l'élagage en temps réel, que les informations affichées restent cohérentes avec l'état de l'algorithme à chaque étape et que l'interface soit simple et claire d'utilisation.

8.2. Tests à réaliser

Interface testée	Action utilisateur	Résultat attendu	Critère de validation
Chargement des données	L'utilisateur clique sur *«importer »* et sélectionner un unique fichier.	Les données sont chargées et confirmées à l'écran via ce message.	Message de confirmation affiché : « Importation validée ».
Contrôle de l'animation	L'utilisateur clique sur les boutons de contrôle de l'animation (*boutons lancer et arrêter)*.	L'animation correspond aux actions effectuées par l'utilisateur.	Seules les actions déclenchées par l'utilisateur sont exécutées.
Pourcentage de branches	L'utilisateur appuie sur les boutons de contrôle.	Le nombre de branches correspond à l'affichage quel que soit l'étape dans lequel se trouve l'animation (en arrêt ou en cours).	Le nombre de branches affiché est cohérent avec l'affichage.
Fichier vide	Le fichier importé par l'utilisateur est vide.	L'application affiche un message d'erreur : « Fichier vide. Animation non disponible. »	Le message d'erreur s'affiche si et seulement si le fichier est vide.
Fichier de données trop court	Le fichier binaire codé sur 16 bits importé par l'utilisateur comporte moins de deux valeurs.	Message d'erreur « Le fichier ne contient pas assez de données. Veuillez importer un fichier binaire codé sur 16 bits	Le message d'erreur s'affiche si et seulement si le fichier contient moins de deux valeurs.

		avec plus de deux valeurs ».	
Téléchargement des fichiers générés	L'utilisateur clique sur « exporter les solutions »	Les fichiers sélectionnés sont téléchargés dans une archive Zip et dans l'emplacement choisi par l'utilisateur.	Un message de confirmation à l'issu de l'opération s'affiche.

8.3. Critères d'acceptation

Nous considérons l'interface comme valide si elle permet un affichage clair et lisible des données, si les interactions avec l'utilisateur via les boutons correspondent à leur rôle défini dans ce cahier, si les messages affichés sont compréhensibles, si la visualisation de l'arbre est fluide et qu'aucune erreur bloquante n'apparaît autre que celles définies dans ce document.

9. Conformité de la documentation

La documentation fournie permet à l'utilisateur d'installer et de comprendre le fonctionnement de l'application. Les informations sont complètes et rédigées en français. La documentation est validée si elle permet une utilisation de l'application sans aide supplémentaire.

10. Critère de validation finale

L'application rendue est considérée comme valide dès lors qu'elle produit un résultat (que ce soit des fichiers ou des messages d'erreur) dans toutes les situations dans lesquelles elle pourrait se trouver.
Les résultats doivent être cohérents avec la situation dans laquelle elle peut se trouver et les fichiers générés doivent être conformes à l'état de l'arbre à la fin de l'animation. De plus, ces derniers doivent être sous format binaire et codés sur 16 bits.

L'application se compose de deux programmes.

Le programme auxiliaire prépare les données.
Il génère à partir de x, son histogramme P ainsi que son homologue sous-quantifié à 1 bit.

Le programme principal est le cœur de l'algorithme de déquantification et génère à partir de x et P, toutes les solutions trouvées. Nécessairement, x se trouve parmi les solutions.

L'application donne la possibilité de directement exécuter le programme principal c'est-à-dire d'importer un dossier composé de xQ et P, de lancer l'animation et de récupérer les résultats.

La validation de notre projet repose sur des jeux de données servant de test et pouvant être regroupé en deux catégories distinctes.
D'une part, nous avons des jeux de données restreints qui permettent de vérifier visuellement et rapidement que nos programmes auxiliaire et principal génèrent bien $xQ, P,$ et les solutions attendues (avec x inclus).

Et d'une autre part, des jeux de données plus complet permettant de valider le passage à l'échelle.

11. Glossaire

Quantification	*Opération irréversible qui consiste à transformer des valeurs continues en valeurs discrètes*
Sous-quantification à 1 bit	*Opération irréversible qui consiste à discrétiser une deuxième fois un jeu de données qui a déjà été quantifié.*
Déquantification	*Opération qui consiste à retrouver la série originale exacte après sa quantification.*
Elagage	*Opération qui consiste à supprimes des branches d'un arbre binaire.*
Série temporelle	*Suite ordonnée de valeurs numériques enregistrées à intervalle régulier de temps.*
Encodage 16 bits	*Représentation d'une valeur numérique sur 16 bits.*
Archive ZIP	*Format de fichier qui a pour objectif de compresser et de regrouper plusieurs fichiers en une seule archive.*

12. Références

KANGNI, D., BENALI, A., & ACHAK, S. (2026). *Cahier des charges L2F2*.

The Julia Language. « Julia 1.12 Documentation ». https://docs.julialang.org/en/v1.12/.

13. Index

arbre binaire, *6*
Archive ZIP, *14*
déquantification, *2*, *4*, *6*
Encodage 16 bits, *14*
GLMakie, *8*
histogramme, *6*, *9*
Julia, *2*, *4*, *6*
quantification, *2*, *4*, *6*
sous-quantification à 1 bit., *6*
ZipFile, *8*